DS HOMEWORK PROGRAMS EXERCISE -8 (2025115100)

1. Simulate BFS and DFS Traversals for a Given Graph

```
#include <stdio.h>

int a[10][10], visited[10], n;

void BFS(int s) {
   int q[10], front = 0, rear = 0, i;

   for (i = 0; i < n; i++)
      visited[i] = 0;

   q[rear++] = s;
   visited[s] = 1;

   while (front < rear) {
      s = q[front++];
      printf("%d ", s);

      for (i = 0; i < n; i++) {
         if (a[s][i] && !visited[i]) {
            q[rear++] = i;
            visited[i] = 1;
         }
      }
   }
}

void DFS(int s) {
   int i;
   visited[s] = 1;
   printf("%d ", s);

   for (i = 0; i < n; i++)
      if (a[s][i] && !visited[i])
         DFS(i);
}

int main() {
   int i, j, s;

   printf("Enter number of vertices: ");
   scanf("%d", &n);

   printf("Enter adjacency matrix:\n");
   for (i = 0; i < n; i++)
```

```
        for (j = 0; j < n; j++)
            scanf("%d", &a[i][j]);

    printf("Enter starting vertex: ");
    scanf("%d", &s);

    printf("BFS: ");
    BFS(s);

    for (i = 0; i < n; i++)
        visited[i] = 0;

    printf("\nDFS: ");
    DFS(s);

    return 0;
}
```

OUTPUT

```
Enter number of vertices: 4
Enter adjacency matrix:
0 1 1 0
1 0 0 1
1 0 0 1
0 1 1 0
Enter starting vertex: 0
BFS: 0 1 2 3
DFS: 0 1 3 2
```

2. Find the Minimum Spanning Tree Using Prim's and Kruskal's Algorithms

A. Prim's Algorithm

```
#include <stdio.h>

#define INF 99999

int main() {
    int a[10][10], vis[10] = {0};
    int n, i, j, edges = 0, min, x = 0, y = 0, cost = 0;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter cost matrix (0 for no edge):\n");
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++) {
            scanf("%d", &a[i][j]);
            if (i != j && a[i][j] == 0)
                a[i][j] = INF;
        }
```

```
  vis[0] = 1;

  while (edges < n - 1) {
    min = INF;

    for (i = 0; i < n; i++)
      if (vis[i])
        for (j = 0; j < n; j++)
          if (!vis[j] && a[i][j] < min) {
            min = a[i][j];
            x = i;
            y = j;
          }

    if (min == INF) {
      printf("Graph is disconnected");
      return 0;
    }

    printf("%d - %d : %d\n", x, y, min);
    cost += min;
    vis[y] = 1;
    edges++;
  }

  printf("Minimum cost = %d", cost);
  return 0;
}
```

OUTPUT

```
Enter number of vertices: 4
Enter cost matrix (0 for no edge):
0 10 6 5
10 0 0 15
6 0 0 4
5 15 4 0
0 - 3 : 5
3 - 2 : 4
0 - 1 : 10
Minimum cost = 19
```

B. Kruskal's Algorithm

```
#include <stdio.h>

struct Edge {
  int u, v, w;
} e[50], temp;
```

```
int parent[20];

int find(int x) {
   while (parent[x] != x)
      x = parent[x];
   return x;
}

int main() {
   int n, m, i, j, cost = 0, count = 0;

   printf("Enter vertices and edges: ");
   scanf("%d%d", &n, &m);

   printf("Enter edges (u v weight):\n");
   for (i = 0; i < m; i++)
      scanf("%d%d%d", &e[i].u, &e[i].v, &e[i].w);

   for (i = 0; i < n; i++)
      parent[i] = i;

   for (i = 0; i < m - 1; i++)
      for (j = 0; j < m - i - 1; j++)
         if (e[j].w > e[j + 1].w) {
            temp = e[j];
            e[j] = e[j + 1];
            e[j + 1] = temp;
         }

   for (i = 0; i < m && count < n - 1; i++) {
      int p = find(e[i].u);
      int q = find(e[i].v);

      if (p != q) {
         printf("%d - %d : %d\n",
               e[i].u, e[i].v, e[i].w);

         parent[p] = q;
         cost += e[i].w;
         count++;
      }
   }

   if (count != n - 1)
      printf("Graph is disconnected");
   else
      printf("Minimum cost = %d", cost);
```

```
    return 0;
}
```

OUTPUT

```
Enter vertices and edges: 4 5
Enter edges (u v weight):
0 1 10
0 2 6
0 3 5
1 3 15
2 3 4
2 - 3 : 4
0 - 3 : 5
0 - 1 : 10
Minimum cost = 19
```

3. Find the Minimum Spanning Tree using Kruskal's Algorithm

```
#include <stdio.h>

struct Edge {
   int u, v, w;
} e[100], t;

int p[50];

int find(int x) {
   while (p[x] != x)
      x = p[x];
   return x;
}

int main() {
   int n, m, i, j, cost = 0, count = 0;

   scanf("%d%d", &n, &m);

   for (i = 0; i < m; i++)
```

```
    scanf("%d%d%d", &e[i].u, &e[i].v, &e[i].w);

for (i = 0; i < n; i++)
    p[i] = i;

for (i = 0; i < m - 1; i++)
    for (j = 0; j < m - i - 1; j++)
        if (e[j].w > e[j + 1].w) {
            t = e[j];
            e[j] = e[j + 1];
            e[j + 1] = t;
        }

for (i = 0; i < m && count < n - 1; i++) {
    int a = find(e[i].u);
    int b = find(e[i].v);

    if (a != b) {
        printf("%d %d %d\n",
               e[i].u, e[i].v, e[i].w);
        p[a] = b;
        cost += e[i].w;
        count++;
    }
}

if (count == n - 1)
    printf("MST cost = %d", cost);
else
    printf("No spanning tree exists");
```

```
    return 0;
}
```

OUTPUT

```
4 5
0 1 10
0 2 6
0 3 5
1 3 15
2 3 4
2 3 4
0 3 5
0 1 10
MST cost = 19
```

4. Write a C program to find the shortest paths using Dijkstra's Algorithm.

```
#include <stdio.h>

#define INF 99999

int main() {
   int a[10][10], d[10], visited[10] = {0};
   int n, s, i, j, u, min;

   scanf("%d", &n);

   for (i = 0; i < n; i++)
      for (j = 0; j < n; j++) {
         scanf("%d", &a[i][j]);
         if (i != j && a[i][j] == 0)
            a[i][j] = INF;
      }
```

```
printf("Enter source: ");
scanf("%d", &s);

for (i = 0; i < n; i++)
   d[i] = a[s][i];

d[s] = 0;
visited[s] = 1;

for (i = 1; i < n; i++) {
   min = INF;
   u = -1;

   for (j = 0; j < n; j++)
      if (!visited[j] && d[j] < min) {
         min = d[j];
         u = j;
      }

   if (u == -1)
      break;

   visited[u] = 1;

   for (j = 0; j < n; j++)
      if (!visited[j] && a[u][j] != INF &&
         d[u] + a[u][j] < d[j])
         d[j] = d[u] + a[u][j];
}
```

```
    printf("Vertex\tDistance\n");
    for (i = 0; i < n; i++) {
        if (d[i] == INF)
            printf("%d\tUnreachable\n", i);
        else
            printf("%d\t%d\n", i, d[i]);
    }

    return 0;
}
```

OUTPUT

```
4
0 4 1 0
4 0 2 5
1 2 0 8
0 5 8 0
0
Enter source: Vertex    Distance
0       0
1       3
      3           1
3       8
```

5.Write a C program for Topological Sorting of a Directed Acyclic Graph (DAG).

```
      #include <stdio.h>

      int main() {
          int a[10][10], indeg[10] = {0};
          int q[10], order[10];
          int n, i, j, front = 0, rear = 0, k = 0, u;
```

```
scanf("%d", &n);

for (i = 0; i < n; i++)
   for (j = 0; j < n; j++) {
      scanf("%d", &a[i][j]);
      if (a[i][j])
         indeg[j]++;
   }

for (i = 0; i < n; i++)
   if (indeg[i] == 0)
      q[rear++] = i;

while (front < rear) {
   u = q[front++];
   order[k++] = u;

   for (j = 0; j < n; j++)
      if (a[u][j] && --indeg[j] == 0)
         q[rear++] = j;
}

if (k != n)
   printf("Graph contains a cycle");
else {
   printf("Topological order: ");
   for (i = 0; i < n; i++)
      printf("%d ", order[i]);
}

return 0;
```

```
    }
    OUTPUT
    4
    0 1 1 0
    0 0 0 1
    0 0 0 1
    0 0 0 0
    Topological order: 0 1 2 3
```

6. Insert Keys into a Hash Table Using Linear Probing

```
#include <stdio.h>

int main() {
   int h[7], a[] = {76, 93, 40, 47, 10, 55};
   int i, k;

   for (i = 0; i < 7; i++)
      h[i] = -1;

   for (i = 0; i < 6; i++) {
      k = a[i] % 7;

      while (h[k] != -1)
         k = (k + 1) % 7;

      h[k] = a[i];
   }

   printf("Index\tValue\n");
   for (i = 0; i < 7; i++)
      printf("%d\t%d\n", i, h[i]);
```

```
    return 0;
}
```

OUTPUT

```
Index   Value
0       93
1       47
2       10
3       55
4       -1
5       40
6       76
```

7. Write a C program for Hashing Using Quadratic Probing.

```
#include <stdio.h>

int main() {
    int h[10], a[] = {23, 43, 13, 27, 33};
    int i, j, k, n = 5, size = 10;

    for (i = 0; i < size; i++)
        h[i] = -1;

    for (i = 0; i < n; i++) {
        k = a[i] % size;
        j = 0;

        while (h[(k + j * j) % size] != -1)
            j++;

        h[(k + j * j) % size] = a[i];
```

```
    }

    printf("Index\tValue\n");
    for (i = 0; i < size; i++)
        printf("%d\t%d\n", i, h[i]);

    return 0;
}
```

OUTPUT

```
Index   Value
0       -1
1       -1
2       -1
3       23
4       43
5       13
6       -1
7       27
8       33
9       -1
```

8. Insert Keys Using Double Hashing

```
#include <stdio.h>

int main() {
    int h[7], a[] = {27, 43, 69, 72};
    int i, j, k, pos;

    for (i = 0; i < 7; i++)
        h[i] = -1;
```

```
    for (i = 0; i < 4; i++) {
        k = a[i];

        for (j = 0; j < 7; j++) {
            pos = (k % 7 + j * (1 + k % 5)) % 7;

            if (h[pos] == -1) {
                h[pos] = k;
                break;
            }
        }
    }

    for (i = 0; i < 7; i++)
        printf("%d: %d\n", i, h[i]);

    return 0;
}
```

OUTPUT

0: 69

1: -1

2: 72

3: 43

4: -1

5: 27

6: -1

9.Prove the hashing statement

Statement: If quadratic probing is used with a prime-sized hash table, a new element can always be inserted when at least half of the table is empty.

Proof:

Let the hash table size be a prime number m.

In quadratic probing, the positions are calculated using:

h(k,i)=(h(k)+i^2) mod m

When the table size is prime, the first m+1/2 probe positions are distinct. Thus, quadratic probing can access at least half of the table's positions.

If at least half of the table is empty, at least one of these distinct positions must be vacant. Therefore, the new element can be inserted successfully.

Hence proved.

10. Implement a Hash Data Structure with Separate Chaining

```
#include <stdio.h>
#include <stdlib.h>

#define SIZE 10

struct Node {
   int data;
   struct Node *next;
};

struct Node *h[SIZE];

void insert(int x) {
   int i = x % SIZE;

   struct Node *n = malloc(sizeof(struct Node));
   n->data = x;
   n->next = h[i];
   h[i] = n;
}

void search(int x) {
   int i = x % SIZE;
   struct Node *t = h[i];
```

```
    while (t) {
        if (t->data == x) {
            printf("Found\n");
            return;
        }
        t = t->next;
    }

    printf("Not found\n");
}

void deleteKey(int x) {
    int i = x % SIZE;
    struct Node *t = h[i], *p = NULL;

    while (t) {
        if (t->data == x) {
            if (p)
                p->next = t->next;
            else
                h[i] = t->next;

            free(t);
            printf("Deleted\n");
            return;
        }

        p = t;
        t = t->next;
    }
```

```
    printf("Not found\n");
}

int main() {
    insert(25);
    insert(15);
    insert(35);

    search(15);
    deleteKey(15);
    search(15);

    return 0;
}
```

Output

Found

Deleted

Not found